

Vivado HLS Design Flow Lab

Objectives

After completing this lab, you will be able to:

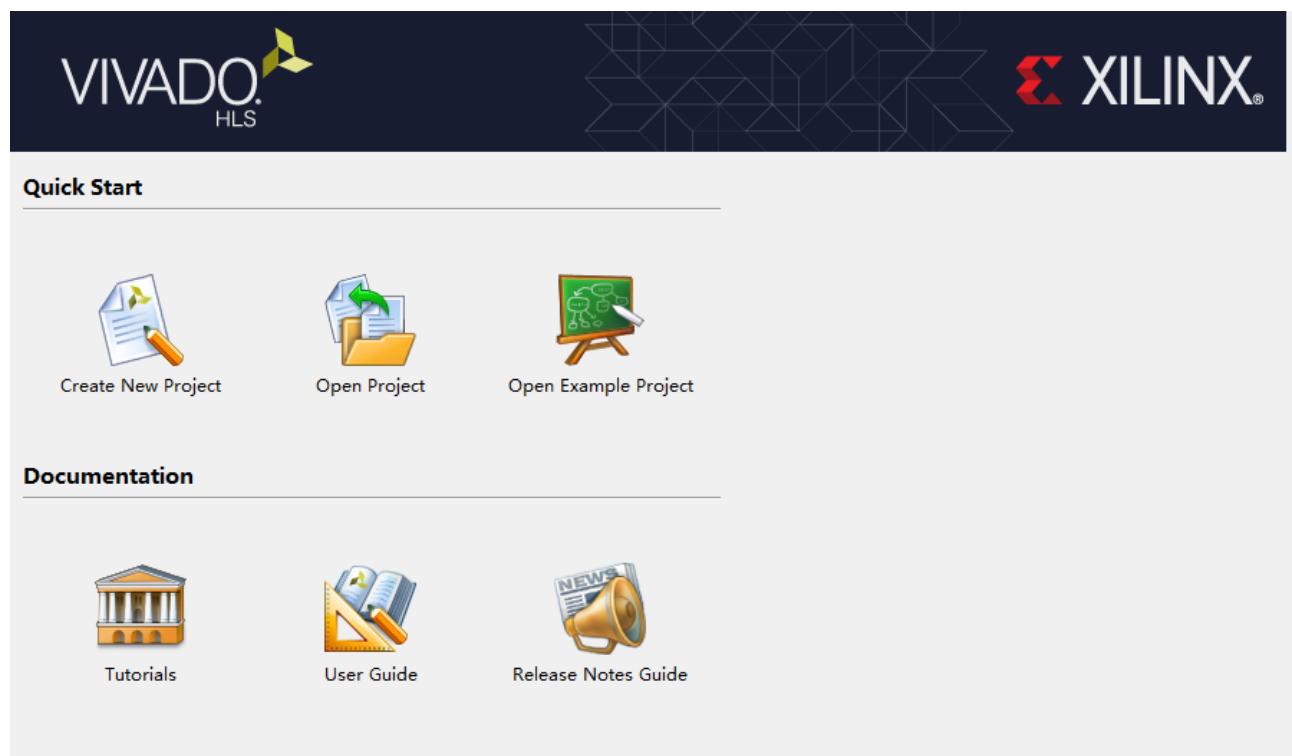
- Create a new project using Vivado HLS in GUI mode
- Simulate a design
- Synthesize a design
- Implement a design
- Perform design analysis using the Analysis capability of Vivado HLS
- Analyze simulator output using Vivado and XSim simulator

Steps

Create a New Project

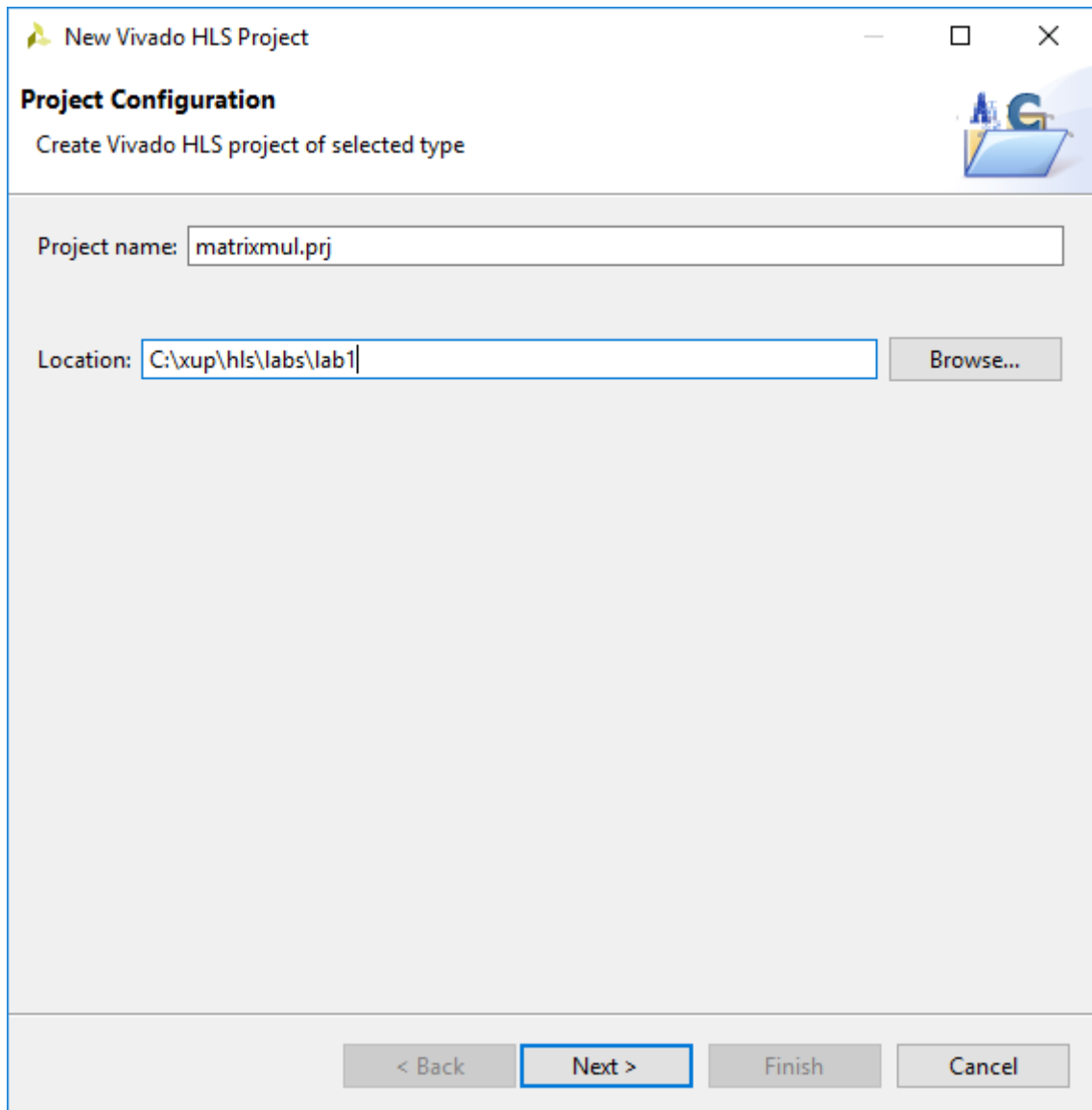
Create a new project in Vivado HLS targeting PYNQ-Z1 or PYNQ-Z2 board

1. Select **Start > Xilinx Design Tools > Vivado HLS 2018.2**



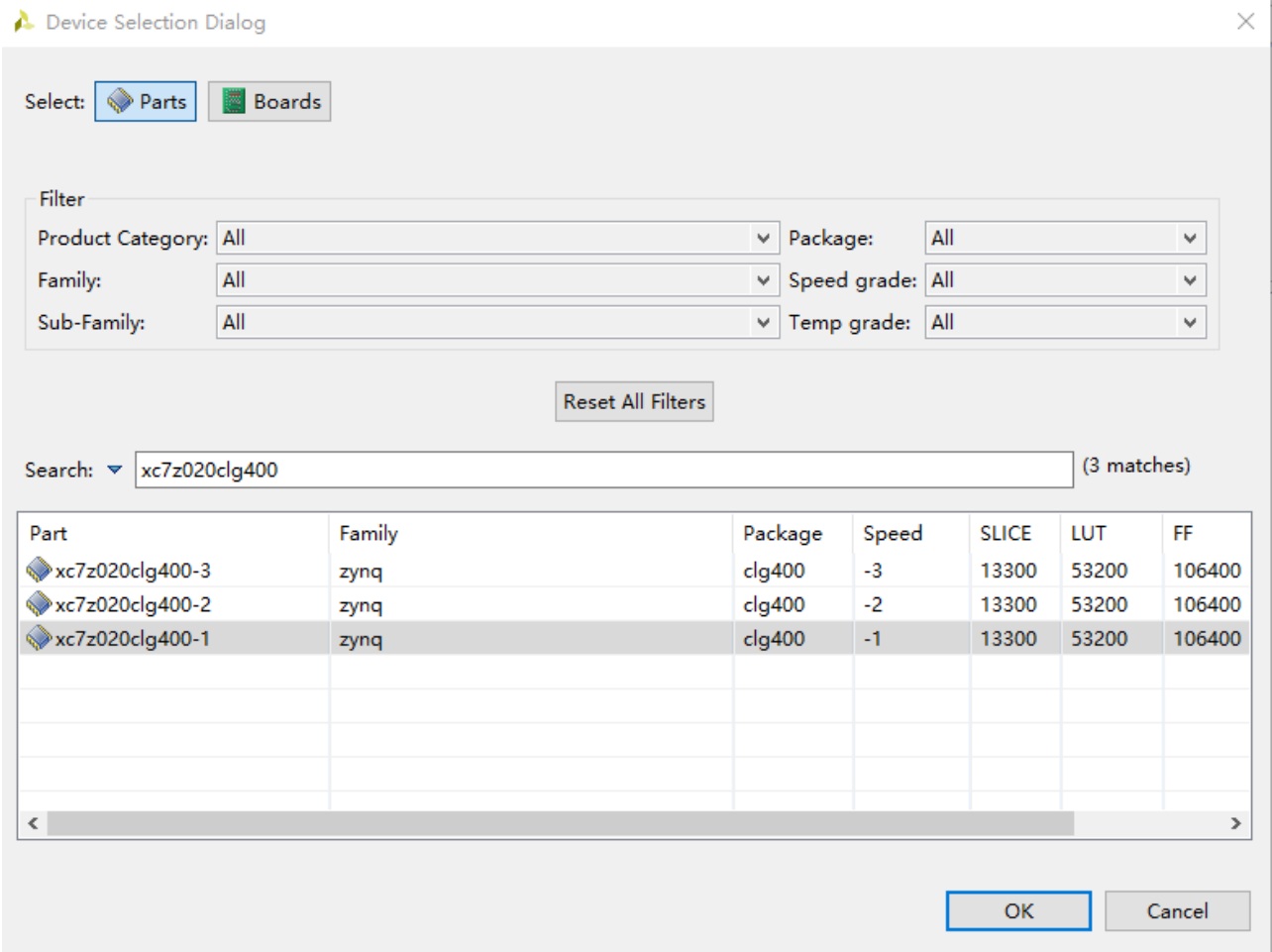
Getting Started view of Vivado-HLS

2. In the Getting Started GUI, click on **Create New Project**. The **New Vivado HLS Project** wizard opens.
3. Click the *Browse...* button of the Location field and browse to **c:\xup\hls\labs\lab1** and then click **OK**.
4. For Project Name, type **matrixmul.prj**



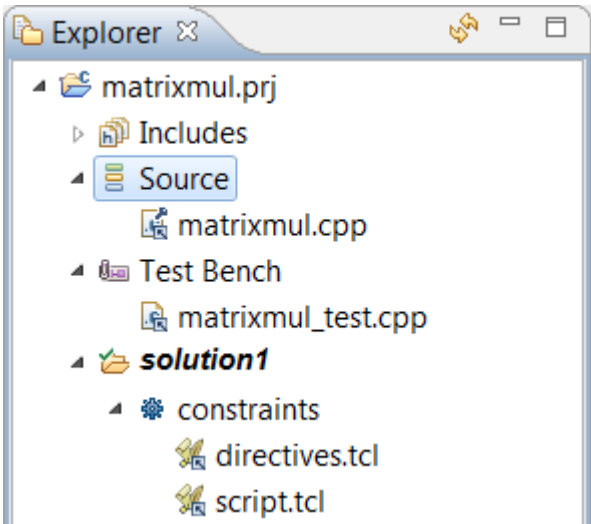
New Vivado HLS Project wizard

5. Click **Next**.
6. In the *Add/Remove Files* window, type **matrixmul** as the Top Function name (the provided source file contains the function, called matrixmul, to be synthesized).
7. Click the *Add Files...* button, select **matrixmul.cpp** file from the **c:\xup\hls\labs\lab1** folder, and then click **Open**.
8. Click **Next**.
9. In the *Add/Remove Files* for the testbench, click the *Add Files...* button, select **matrixmul_test.cpp** file from the **c:\xup\hls\labs\lab1** folder and click **Open**.
10. Select the **matrixmul_test.cpp** in the files list window and click the *Edit CFLAG...* button, type - **DHW_COSIM**, and click **OK**. (This defines a compiler flag that will be used later.)
11. Click **Next**.
12. In the *Solution Configuration* page, leave **Solution** Name field as *solution1* and set the clock period as 10.
13. Click the ... button of the *Part Selection* section.
14. In the *Device Selection* Dialog page, select **Parts Specify** field, and select the following filters to select the **xc7z020clg400-1** part and click **OK**.



Using Parts Specify option in Part Selection Dialog

15. Click **Finish**. You will see the created project in the *Explorer* view. Expand various sub-folders to see the entries under each sub-folder.



Explorer Window

16. Double-click on the **matrixmul.cpp** under the source folder to open its content in the information pane.

```

67#include "matrixmul.h"
68
69void matrixmul(
70    mat_a_t a[MAT_A_ROWS][MAT_A_COLS],
71    mat_b_t b[MAT_B_ROWS][MAT_B_COLS],
72    result_t res[MAT_A_ROWS][MAT_B_COLS])
73{
74    // Iterate over the rows of the A matrix
75    Row: for(int i = 0; i < MAT_A_ROWS; i++) {
76        // Iterate over the columns of the B matrix
77        Col: for(int j = 0; j < MAT_B_COLS; j++) {
78            // Do the inner product of a row of A and col of B
79            res[i][j] = 0;
80            Product: for(int k = 0; k < MAT_B_ROWS; k++) {
81                res[i][j] += a[i][k] * b[k][j];
82            }
83        }
84    }
85}

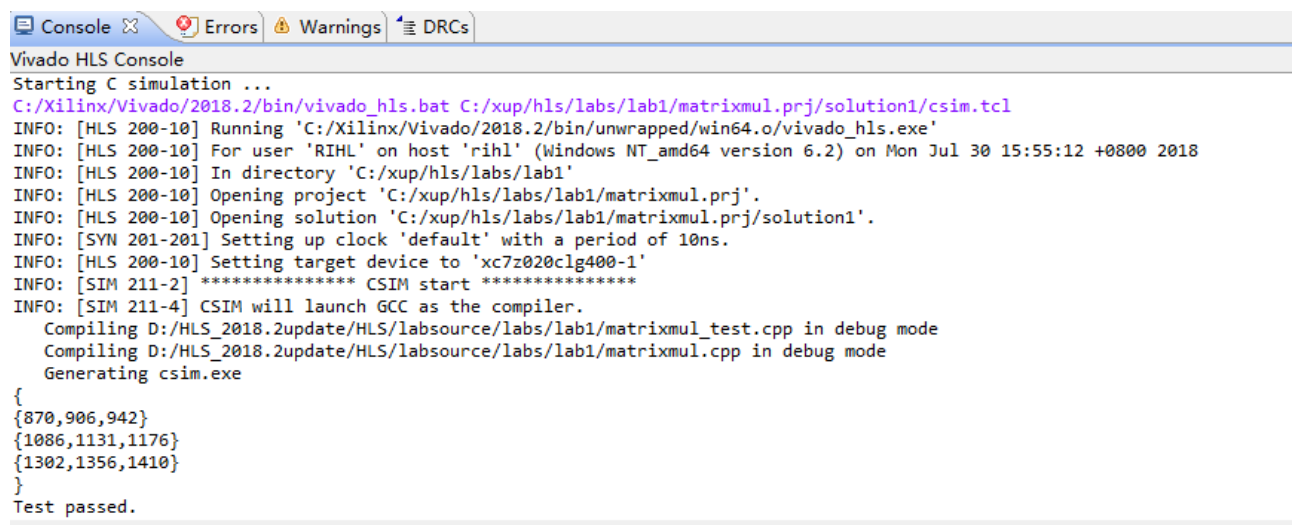
```

The Design under consideration

It can be seen that the design is a matrix multiplication implementation, consisting of three nested loops. The Product loop is the inner most loop performing the actual Matrix elements product and sum. The Col loop is the outer-loop which feeds the next column element data with the passed row element data to the Product loop. Finally, Row is the outer-most loop. The `res[i][j]=0` (line 79) resets the result every time a new row element is passed and new column element is used.

Run C Simulation

1. Select **Project > Run C Simulation**, and click **OK** in the C Simulation Dialog window. The files will be compiled and you will see the output in the Console window.



```

Vivado HLS Console
Starting C simulation ...
C:/Xilinx/Vivado/2018.2/bin/vivado_hls.bat C:/xup/hls/labs/lab1/matrixmul.prj/solution1/csim.tcl
INFO: [HLS 200-10] Running 'C:/Xilinx/Vivado/2018.2/bin/unwrapped/win64.o/vivado_hls.exe'
INFO: [HLS 200-10] For user 'RIHL' on host 'rihl' (Windows NT_amd64 version 6.2) on Mon Jul 30 15:55:12 +0800 2018
INFO: [HLS 200-10] In directory 'C:/xup/hls/labs/lab1'
INFO: [HLS 200-10] Opening project 'C:/xup/hls/labs/lab1/matrixmul.prj'.
INFO: [HLS 200-10] Opening solution 'C:/xup/hls/labs/lab1/matrixmul.prj/solution1'.
INFO: [SYN 201-201] Setting up clock 'default' with a period of 10ns.
INFO: [HLS 200-10] Setting target device to 'xc7z020clg400-1'
INFO: [SIM 211-2] ***** CSIM start *****
INFO: [SIM 211-4] CSIM will launch GCC as the compiler.
Compiling D:/HLS_2018.2update/HLS/labs/labs/lab1/matrixmul_test.cpp in debug mode
Compiling D:/HLS_2018.2update/HLS/labs/labs/lab1/matrixmul.cpp in debug mode
Generating csim.exe
{
{870,906,942}
{1086,1131,1176}
{1302,1356,1410}
}
Test passed.

```

Program output

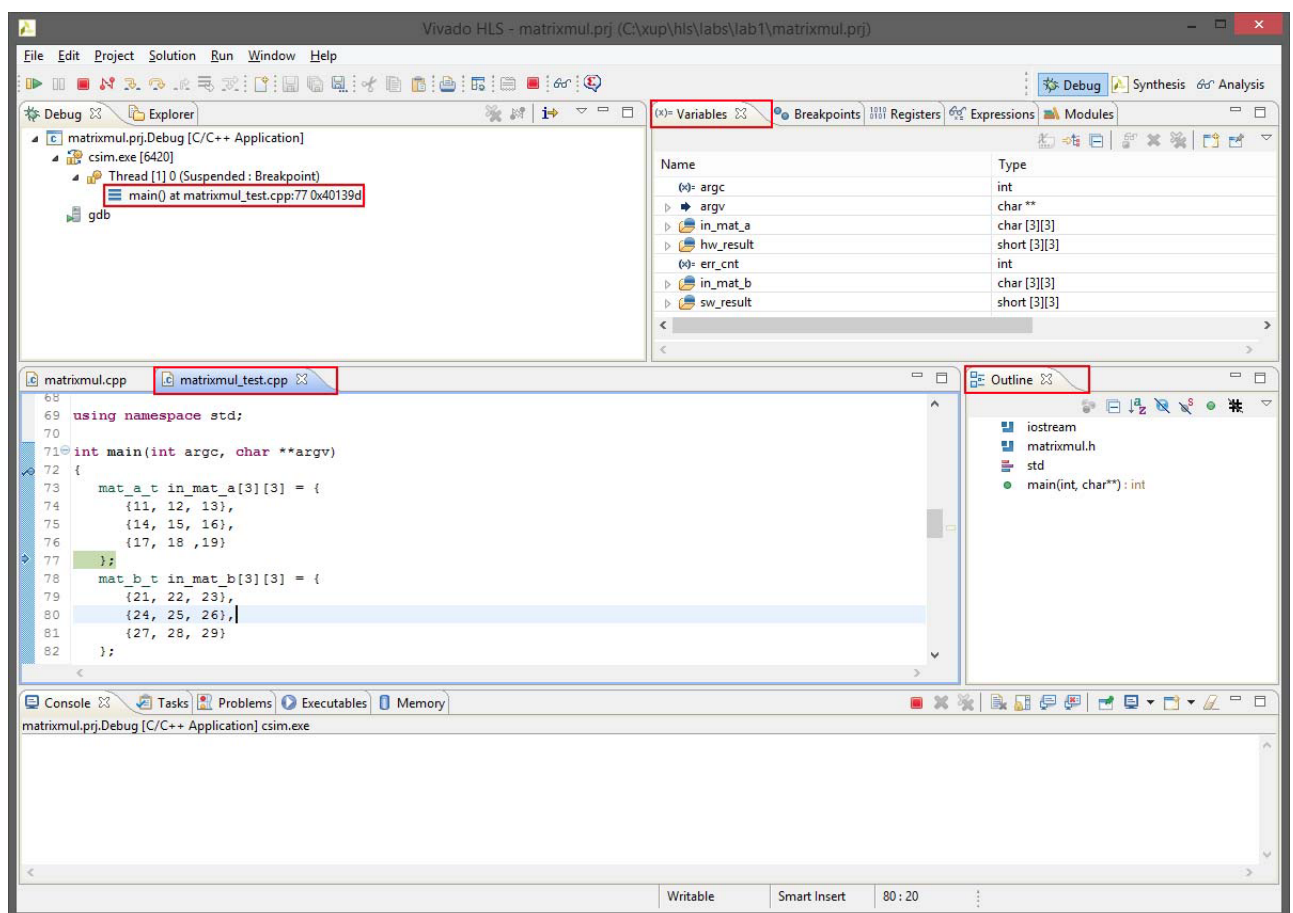
2. Double-click on **matrixmul_test.cpp** under **testbench** folder in the Explorer to see the content. You should see two input matrices initialized with some values and then the code that executes the

algorithm. If `HW_COSIM` is defined (as was done during the project set-up) then the **matrixmul** function is called and compares the output of the computed result with the one returned from the called function, and prints **Test passed** if the results match. If `HW_COSIM` had not been defined then it will simply output the computed result and not call the **matrixmul** function.

Run Debugger

Run the application in debugger mode and understand the behavior of the program.

1. Select **Project > Run C Simulation**. Select the **Launch Debugger** option and click **OK**. The application will be compiled with `-g` option to include the debugging information, the compiled application will be invoked, and the debug perspective will be opened automatically.
2. The Debug perspective will show the **matrixmul_test.cpp** in the source view, **argc** and **argv** variables defined in the *Variables* view, objects which are in the current scope displayed in *Outline* view, thread created and the program suspended at the `main()` function entry point in the *Debug* view.



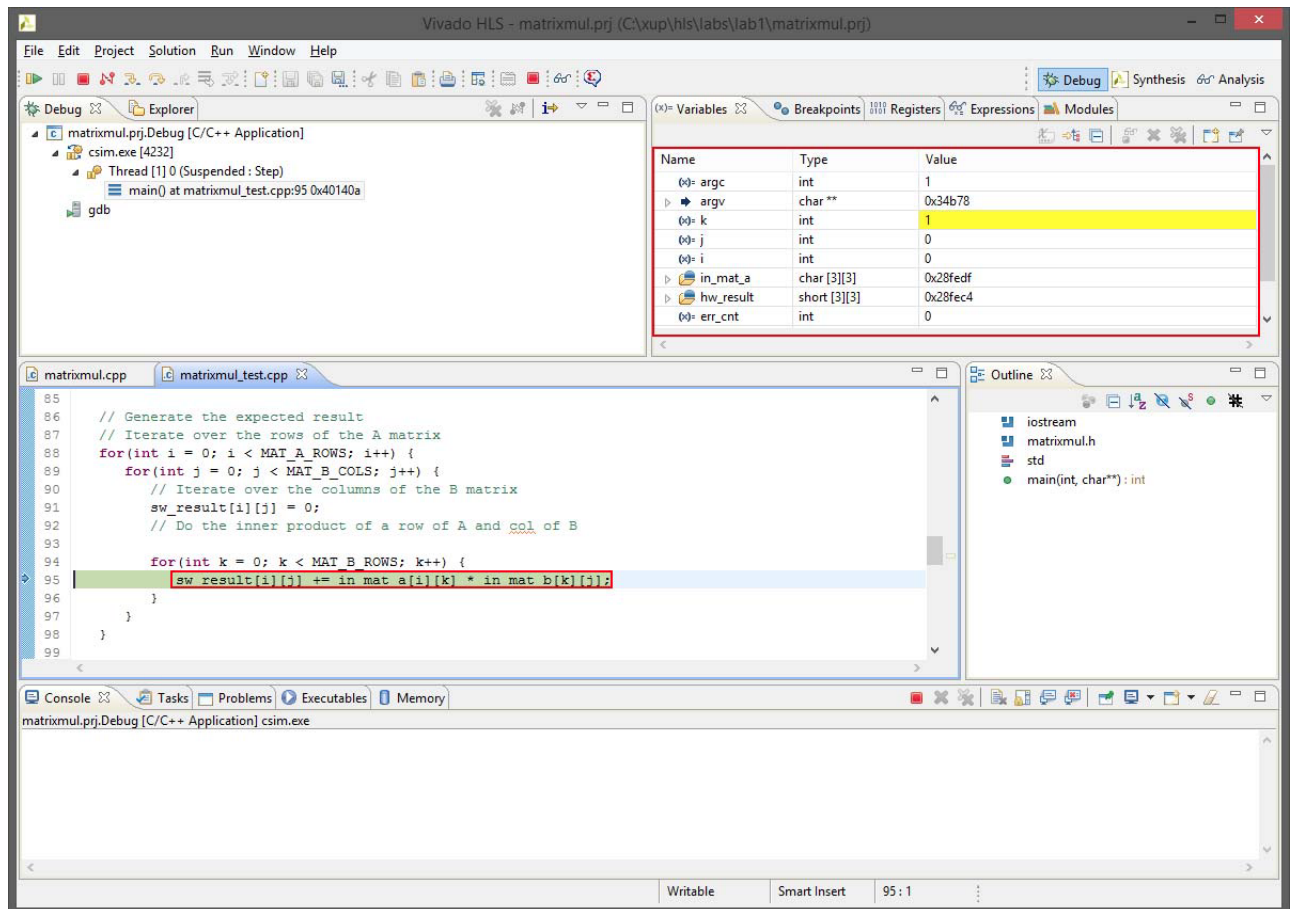
A Debug perspective

3. Scroll-down in the *source* view, and double-click in the blue margin at line 105 where it is about to output `"{"` in the output console window. This will set a break-point at line 105. The breakpoint is marked with a blue circle, and a tick.

```
104 // Print result matrix
105 cout << "{" << endl;
```

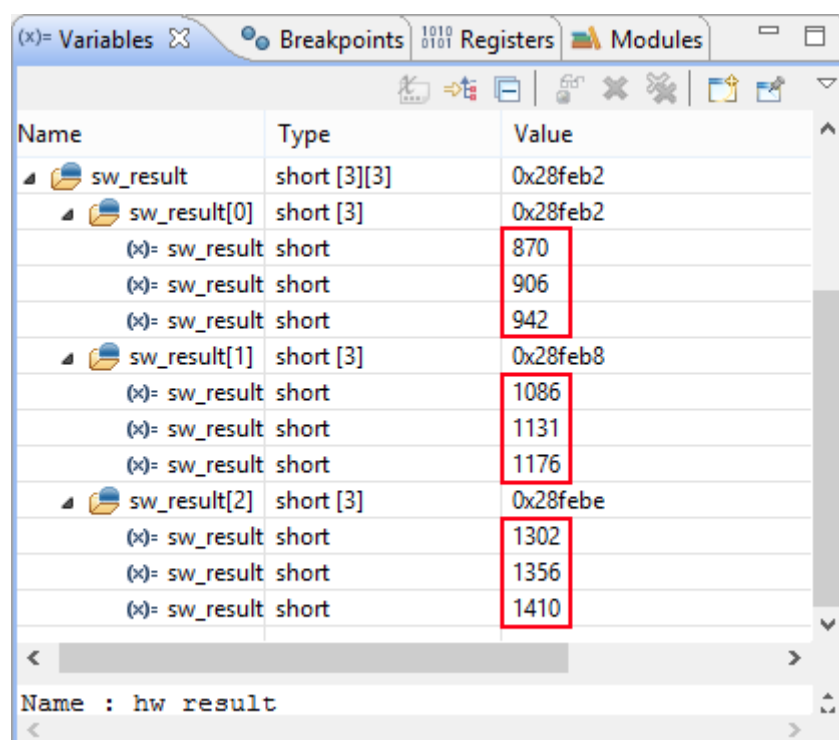
4. Similarly, set a breakpoint at line 101 in the `matrixmul()` function.

5. Using the **Step Over** (F6) button several times, observe the execution progress, and observe the variable values updating, as well as computed software result.



Debugger's intermediate output view

6. Now click the **Resume** button or F8 to complete the software computation and stop at line 101.
7. Observe the following computed software result in the variables view.



Software computed result

8. Click on the **Step Into** (F5) button to traverse into the **matrixmul** module, the one that we will synthesize, and observe that the execution is paused on line 75 of the module.
9. Using the **Step Over** (F6) several times, observe the computed results. Once satisfied, you can use the **Step Return** (F7) button to return from the function.
10. The program execution will suspend at line 105 as we had set a breakpoint. Observe the software and hardware (function) computed results in the Variables view.

Name	Type	Value
hw_result[0]	short [3]	0x28fec4
(x)= hw_result	short	870
(x)= hw_result	short	906
(x)= hw_result	short	942
hw_result[1]	short [3]	0x28feca
hw_result[2]	short [3]	0x28fed0
(x)= err_cnt	int	0
in_mat_b	char [3][3]	0x28fed6
sw_result	short [3][3]	0x28feb2
sw_result[0]	short [3]	0x28feb2
(x)= sw_result	short	870
(x)= sw_result	short	906
(x)= sw_result	short	942

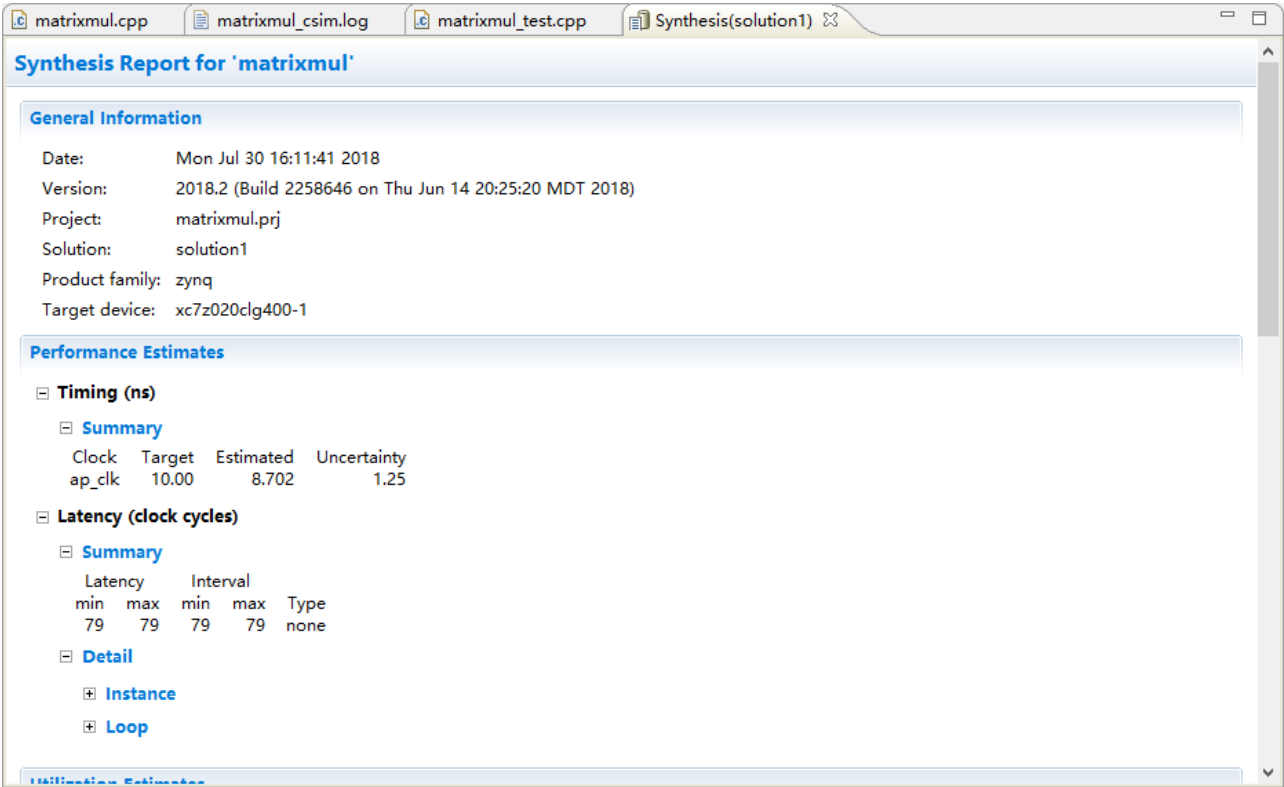
Computed results

11. Set a breakpoint on line 134 (return err_cnt;), and click on the **Resume** button. The execution will continue until the breakpoint is encountered. The console window will show the results as seen earlier (Figure 7).
12. Press the **Resume** button or **Terminate** button to finish the debugging session.

Synthesize the Design

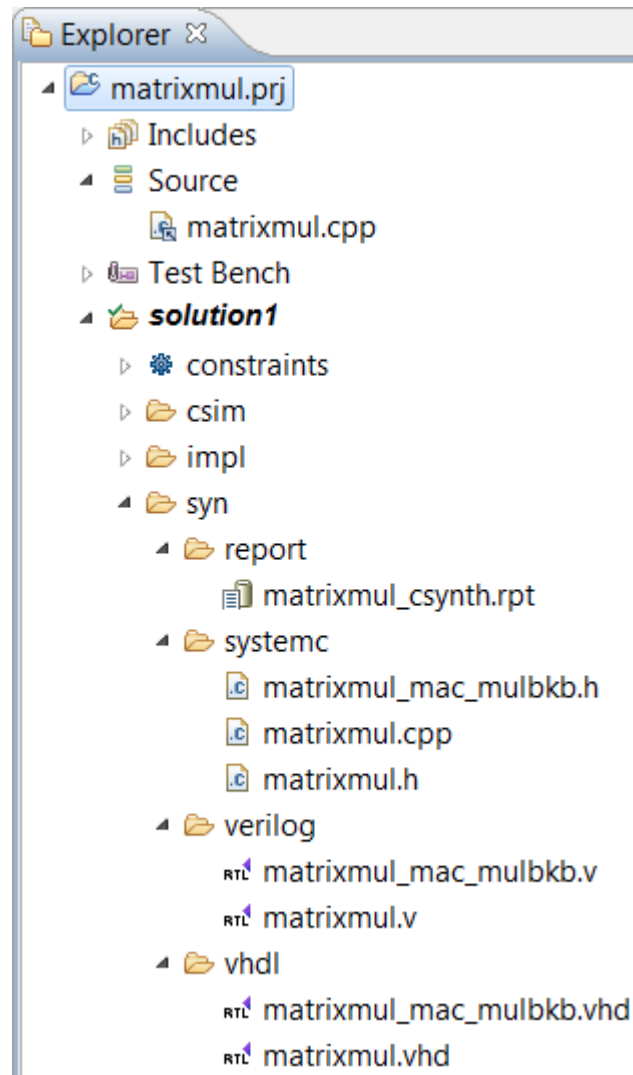
Switch to Synthesis view and synthesize the design with the defaults. View the synthesis results and answer the question listed in the detailed section of this step.

1. Switch to the *Synthesis* view by clicking **Synthesis** button on the tools bar.
2. Select **Solution > Run C Synthesis > Active Solution** to start the synthesis process.
3. When the synthesis process is completed, the synthesis results will be displayed along with the Outline pane. Using the Outline pane, one can navigate to any part of the report with a simple click.



Report view after synthesis is completed

4. If you expand **solution1** in Explorer, several generated files including report files will become accessible.



Explorer view after the synthesis process

Note that when the ****syn**** folder under the Solution1 folder is expanded in the Explorer view, it will show ***report**, **systemC**, **verilog**,* and ***vhdl*** sub-folders under which report files, and generated source (vhdl, verilog, header, and cpp) files. By double-clicking any of these entries will open the corresponding file in the information pane. Also note that if the target design has hierarchical functions, reports corresponding to lower-level functions are also created.

5. The *Synthesis Report* shows the performance and resource estimates as well as estimated latency in the design.
6. Using scroll bar on the right, scroll down into the report and answer the following question.

Question 1 Answer the following question:

Estimated clock period:

Worst case latency:

Number of DSP48E used:

Number of FFs used:

Number of LUTs used:

7. The report also shows the top-level interface signals generated by the tools.

Interface						
Summary						
RTL Ports	Dir	Bits	Protocol	Source Object	C Type	
ap_clk	in	1	ap_ctrl_hs	matrixmul	return value	
ap_rst	in	1	ap_ctrl_hs	matrixmul	return value	
ap_start	in	1	ap_ctrl_hs	matrixmul	return value	
ap_done	out	1	ap_ctrl_hs	matrixmul	return value	
ap_idle	out	1	ap_ctrl_hs	matrixmul	return value	
ap_ready	out	1	ap_ctrl_hs	matrixmul	return value	
a_address0	out	4	ap_memory	a	array	
a_ce0	out	1	ap_memory	a	array	
a_q0	in	8	ap_memory	a	array	
b_address0	out	4	ap_memory	b	array	
b_ce0	out	1	ap_memory	b	array	
b_q0	in	8	ap_memory	b	array	
res_address0	out	4	ap_memory	res	array	
res_ce0	out	1	ap_memory	res	array	
res_we0	out	1	ap_memory	res	array	
res_d0	out	16	ap_memory	res	array	

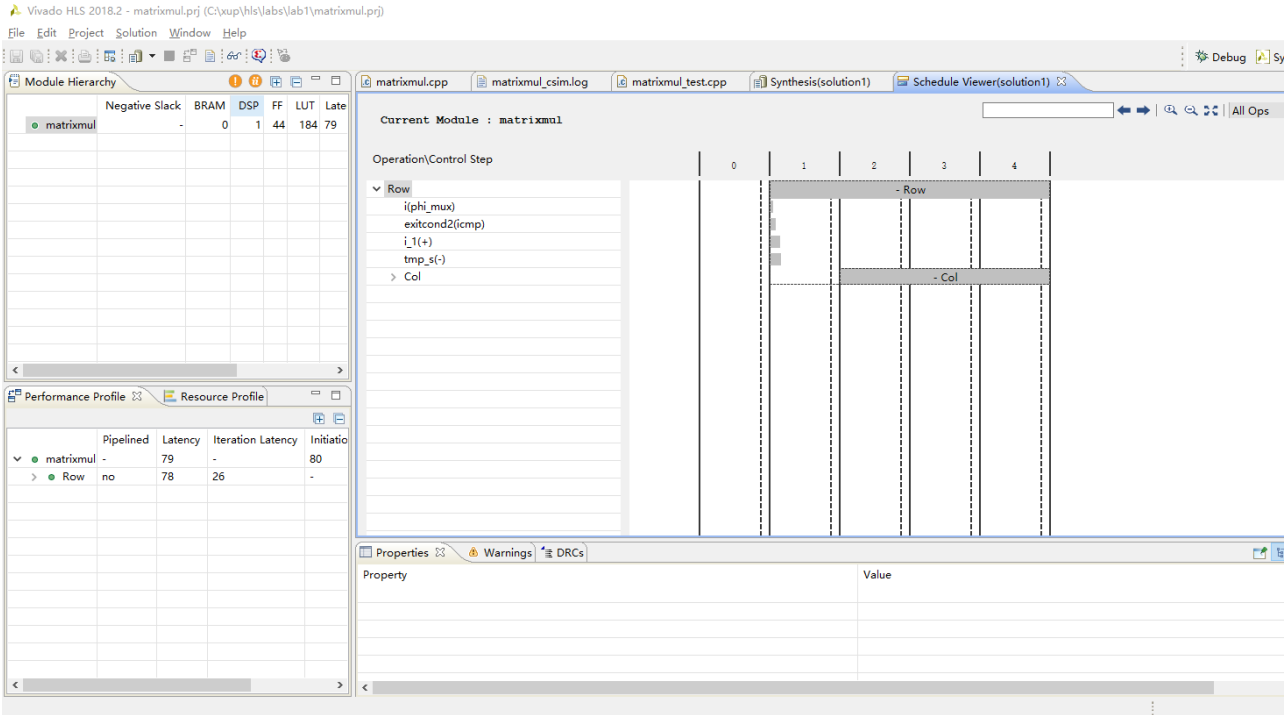
Generated interface signals

You can see ap_clk, ap_rst and ap_idle and ap_ready control signals are automatically added to the design by default. These signals are used as handshaking signals to indicate when the design is ready to take the next computation command (ap_ready), when the next computation is started (ap_start), and when the computation is completed (ap_done). Other signals are generated based on the input and output signals in the design and their default or specified interfaces.

Analyze using Analysis Perspective

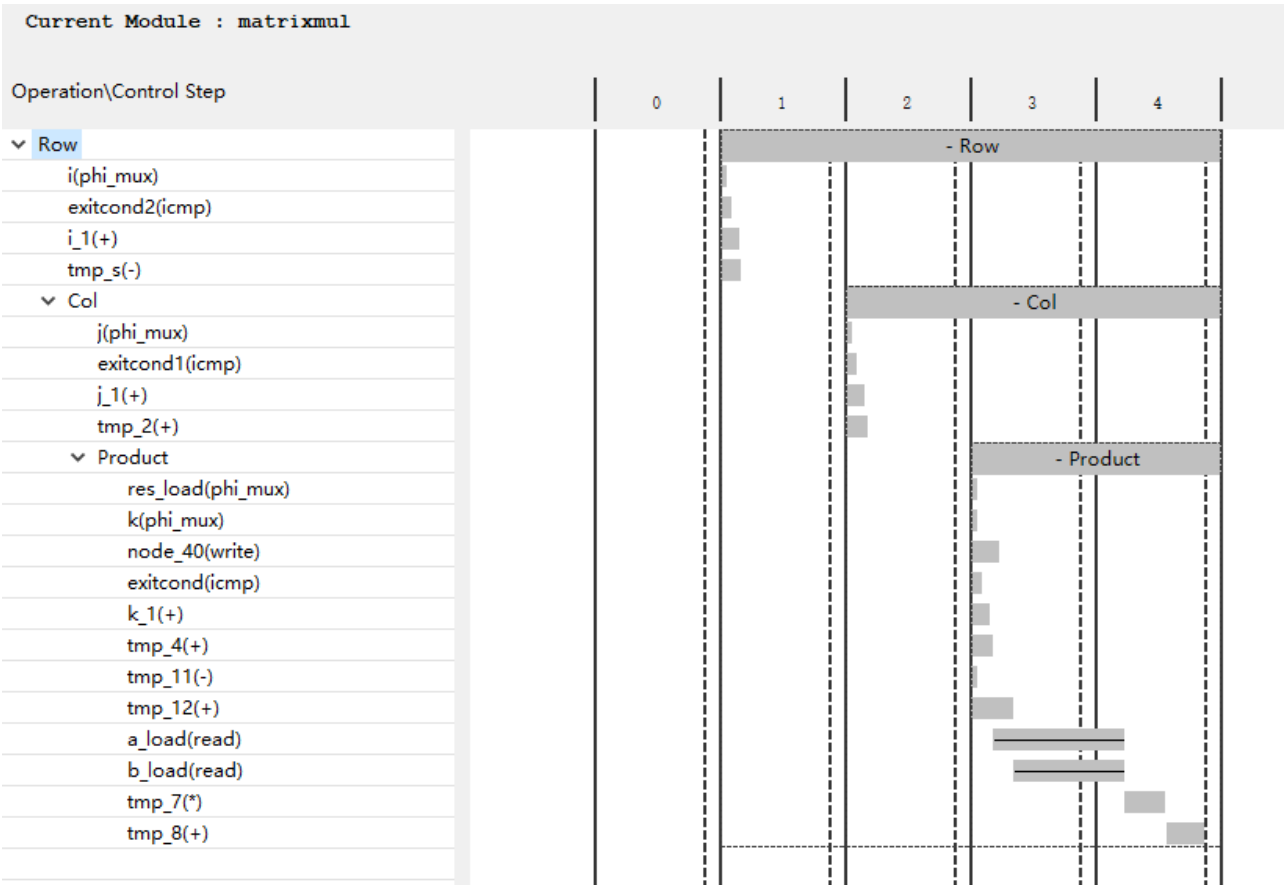
Switch to the Analysis Perspective and understand the design behavior.

1. Select **Solution > Open Analysis Perspective** or click on *Analysis* button on tools bar to open the analysis viewer. The Analysis perspective consists of 4 panes as shown below. Note that the module and loops hierarchies are displayed unexpanded by default. The **Module Hierarchy** pane shows both the performance and area information for the entire design and can be used to navigate through the hierarchy. The **Performance Profile** pane is visible and shows the performance details for this level of hierarchy. The information in these two panes is similar to the information reviewed earlier in the synthesis report. The **Schedule Viewer** is also shown in the right-hand side pane. This view shows how the operations in this particular block are scheduled into clock cycles.



Analysis perspective

2. Click on '>' of loop Row to expand, and then similarly click on sub-loops **Col** and **Product** to fully expand the loop hierarchy.



Performance matrix showing top-level Row operation

From this we can see that in the first state (C1) of the Row the loop exit condition is checked and there is an add operation performed. This addition is likely the counter to count the loop iterations, and we

can confirm this.

3. Select the block for the **adder (i_1(+))** in state C1, right-click and select **Goto Source**. The source code pane will be opened, highlighting line 75 where the Row loop index is being tested and incremented. In the next state (C2) it starts to execute the Col loop.

Current Module : matrixmul

Operation/Control Step

- Row
 - i(phi_mux)
 - exitcond2(icmp)
 - i_1(+)**
 - tmp_s(-)
- Col
 - j(phi_mux)
 - exitcond1(icmp)
 - j_1(+)
 - tmp_2(+)
- Product
 - res_load(phi_mux)
 - k(phi_mux)
 - node_40(write)
 - exitcond(icmp)
 - k_1(+)
 - tmp_4(+)
 - tmp_11(-)
 - tmp_12(+)
 - a_load(read)
 - b_load(read)
 - tmp_7(*)
 - tmp_8(+)

Properties Warnings DRCs C Source

File: D:\HLS_2018.2update\HLS\labsource\labs\lab1\matrixmul.cpp

```

73 {
74 // Iterate over the rows of the A matrix
75 Row: for(int i = 0; i < MAT_A_ROWS; i++) {
76 // Iterate over the columns of the B matrix
77 Col: for(int j = 0; j < MAT_B_COLS; j++) {
78 // Do the inner product of a row of A and col of B
79 res[i][j] = 0;
80 Product: for(int k = 0; k < MAT_B_ROWS; k++) {
81 res[i][j] += a[i][k] * b[k][j];
82 }
83 }
84 }

```

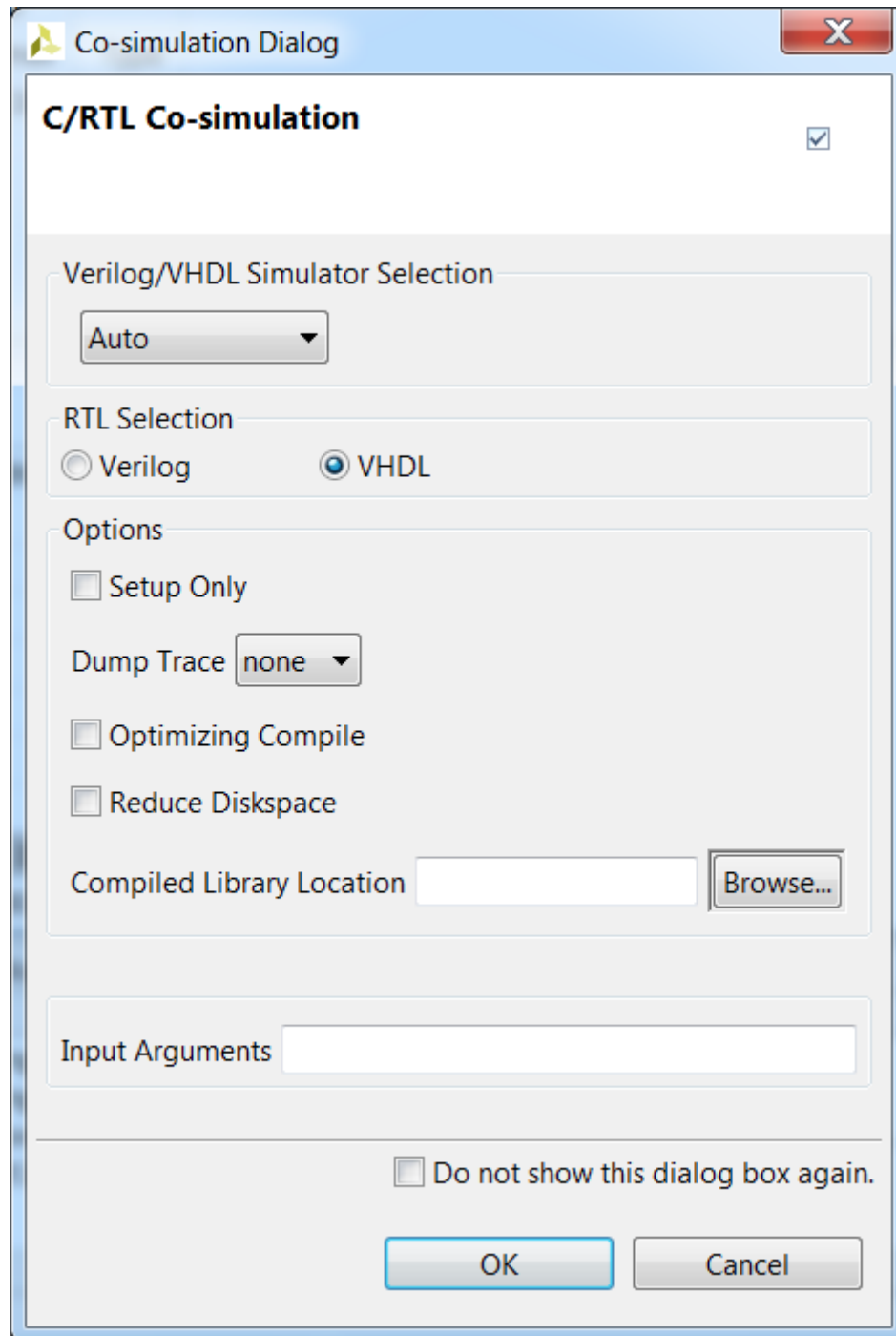
Cross probing into the source file

4. Click on the cell (**tmp 2(+)**) in the Col loop to see the source code highlighting (line 79) update.
5. Expand the Performance Profile hierarchy and note iteration latencies, Trip counts, and overall latencies for each of the nested loops.

	Pipelined	Latency	Iteration Latency	Initiation Interval	Trip count
matrixmul	-	79	-	80	-
Row	no	78	26	-	3
Col	no	24	8	-	3
	no	6	2	-	3

The Performance Profile output

- Click next to the **matrixmul** entry in the Module Hierarchy and observe that the entry is not expanded, since there are no lower-level functions defined in the design.
- Select the **Resource Profile** tab and observe various resources and where they have been used. You can expand Expressions and Registers sections to see how the resources are being used by which operations.



A C/RTL Co-simulation Dialog

3. Click **OK** to run the VHDL simulation. The C/RTL Co-simulation will run, generating and compiling several files, and then simulating the design. It goes through three stages. First, the VHDL test bench is executed to generate input stimuli for the RTL design. Second, an RTL test bench with newly generated input stimuli is created and the RTL simulation is then performed. Finally, the output from the RTL is re-applied to the VHDL test bench to check the results. In the console window you can see the progress and also a message that the test is passed. This eliminates writing a separate testbench for the synthesized design.

```
***** xsim v2018.2 (64-bit)
**** SW Build 2258646 on Thu Jun 14 20:03:12 MDT 2018
**** IP Build 2256618 on Thu Jun 14 22:10:49 MDT 2018
** Copyright 1986-2018 Xilinx, Inc. All Rights Reserved.

source xsim.dir/matrixmul/xsim_script.tcl
# xsim {matrixmul} -autoloadwcfg -tclbatch {matrixmul.tcl}
Vivado Simulator 2018.2
Time resolution is 1 ps
source matrixmul.tcl
## run all
Note: simulation done!
Time: 975 ns Iteration: 1 Process: /apath_matrixmul_top/generate_sim_done_proc File: C:/xup/hls/labs/lab1/matrixmul.prj/solution1/sim/vhdl/matrixmul.autotb.vhd
Failure: NORMAL EXIT (note: failure is to force the simulator to stop)
Time: 975 ns Iteration: 1 Process: /apath_matrixmul_top/generate_sim_done_proc File: C:/xup/hls/labs/lab1/matrixmul.prj/solution1/sim/vhdl/matrixmul.autotb.vhd
$finish called at time : 975 ns
## quit
INFO: [Common 17-206] Exiting xsim at Mon Jul 30 17:16:09 2018...
INFO: [COSIM 212-316] Starting C post checking ...
{
{870,906,942}
{1086,1131,1176}
{1302,1356,1410}
}
Test passed.
INFO: [COSIM 212-1090] *** C/RTL co-simulation finished: PASS ***
INFO: [COSIM 212-211] It is measurable only when transaction number is greater than 1 in RTL simulation. Otherwise, they will be marked as all NA. If user wants to calculate them, please make sure they
Finished C/RTL cosimulation.
```

Console view showing simulation progress

4. Once the simulation verification is completed, the simulation report tab will open showing the results. The report indicates if the simulation passed or failed. In addition, the report indicates the measured latency and interval. Since we have selected only VHDL, the result shows the latencies and interval (initiation) which indicates after how many clock cycles later the next input can be provided. Since the design is not pipelined, it will be latency+1 clock cycles.

Cosimulation Report for 'matrixmul'

Result

		Latency			Interval		
RTL	Status	min	avg	max	min	avg	max
VHDL	Pass	79	79	79	NA	NA	NA
Verilog	NA	NA	NA	NA	NA	NA	NA

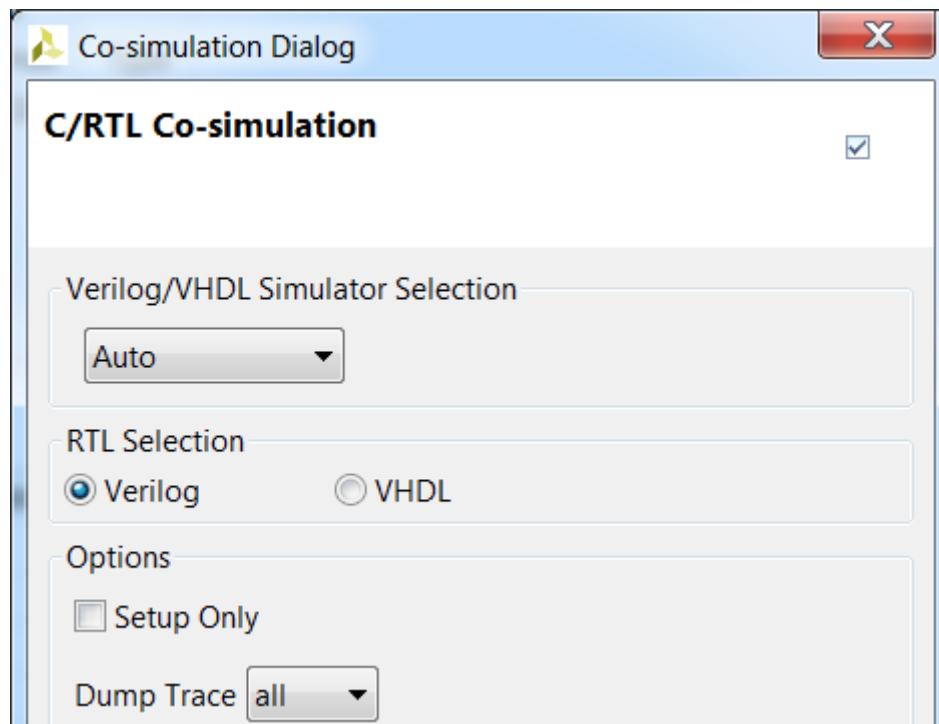
Export the report(.html) using the [Export Wizard](#)

Co-simulation results

Viewing Simulation Results in Vivado

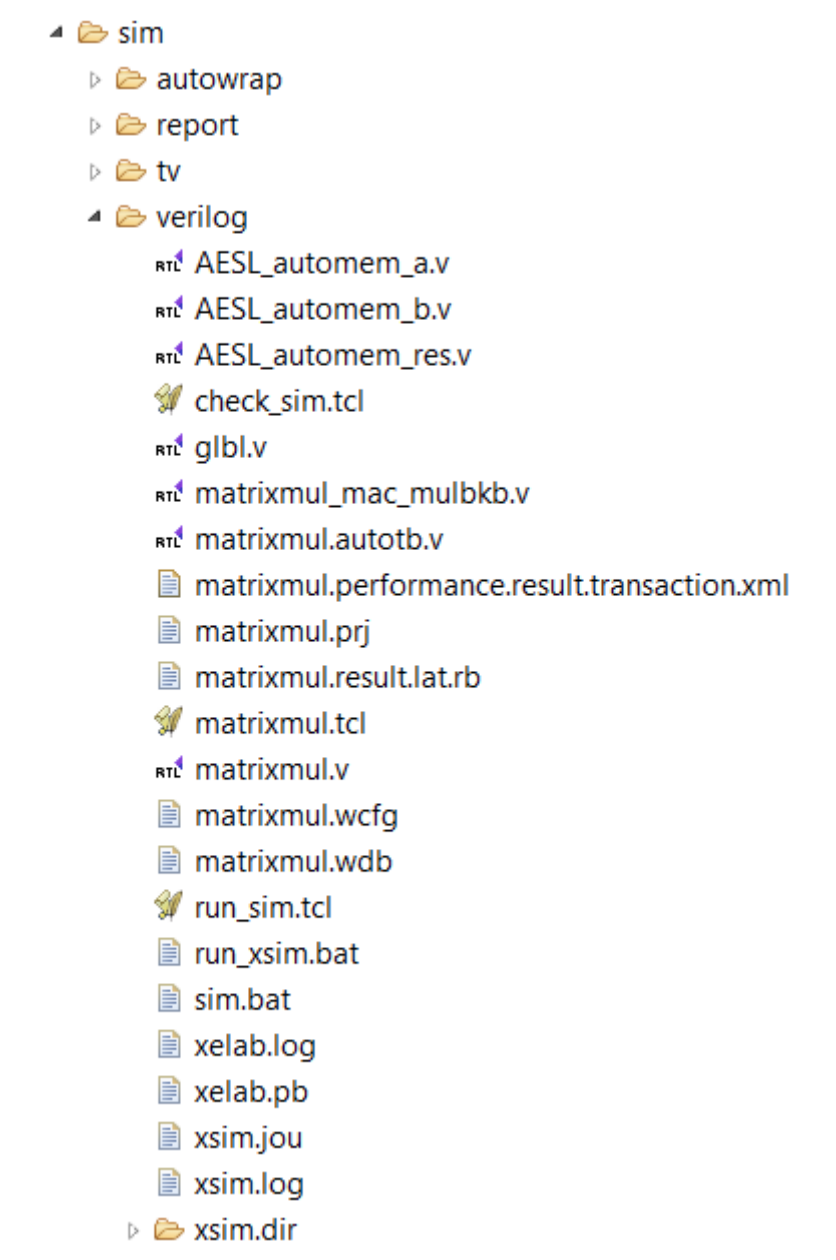
Run Verilog simulation with Dump Trace option selected.

- 1. Select **Solution > Run C/RTL Co-simulation** or click on the  button in the *Synthesis* view to open the dialog box so the desired simulations can be run.
- 2. Click on the **Verilog RTL** Selection option, leaving Verilog/VHDL Simulator Section option to Auto. Optionally, you can click on the drop-down button and select the desired simulator from the available list of XSim, ISim, ModelSim, and Riviera.
- 3. Select **All** for the *Dump Trace* option and click **OK**.



Setting up for Verilog simulation and dump trace

When RTL verification completes the co-simulation report automatically opens showing the Verilog simulation has passed (and the measured latency and interval). In addition, because the Dump Trace option was used and Verilog was selected, two trace files entries can be seen in the Verilog simulation directory.



Explorer view after the Verilog RTL co-simulation run

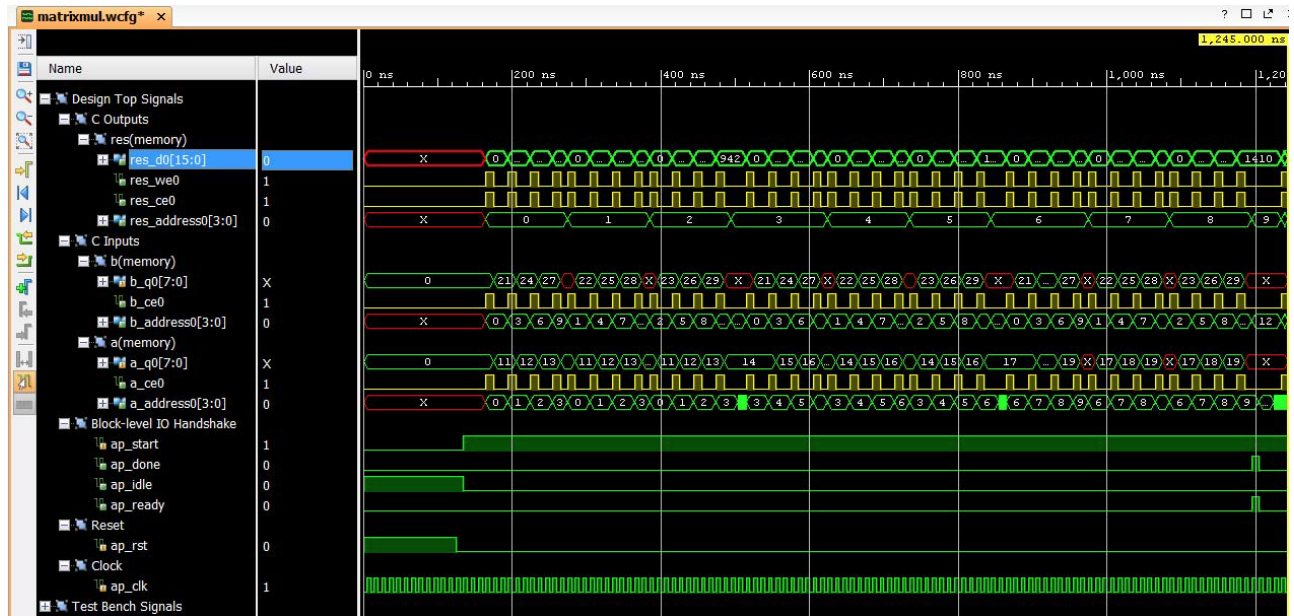
The Co-simulation report shows the test was passed for Verilog along with latency and Interval results.

Cosimulation Report for 'matrixmul'							
Result							
		Latency			Interval		
RTL	Status	min	avg	max	min	avg	max
VHDL	NA	79	79	79	NA	NA	NA
Verilog	Pass	79	79	79	NA	NA	NA

Cosimulation report

Analyze the dumped traces.

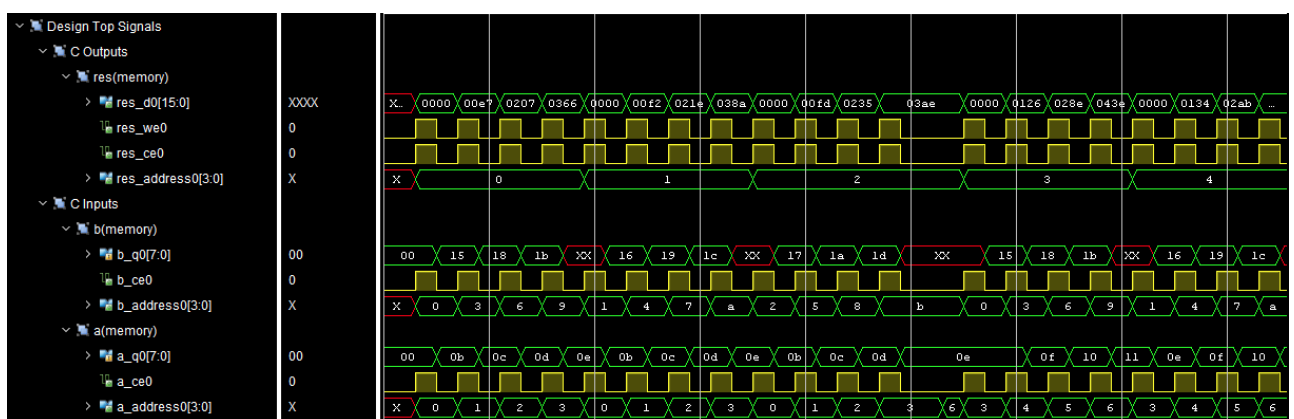
1. Click on the  button on tools bar to open the wave viewer. This will start Vivado 2018.2 and open the wave viewer.
2. Click on the zoom fit tool button () to see the entire simulation of one iteration.
3. Select `a_address0` in the waveform window, right-click and select **Radix > Unsigned Decimal**. Similarly, do the same for `b_address0` and `res_address0` signals.
4. Similarly, set the `a_q0`, `b_q0`, and `res_d0` radix to **Signed Decimal**. You should see the output similar to shown below.



Full waveform showing iteration worth simulation

Note that as soon as `ap_start` is asserted, `ap_idle` has been de-asserted indicating that the design is in computation mode. The `ap_idle` signal remains de-asserted until `ap_done` is asserted, indicating completion of the process. This indicates 79 clock cycles latency.

5. Using the *Zoom In* button, view area of ~160 ns and ~550 ns.



Zoomed view

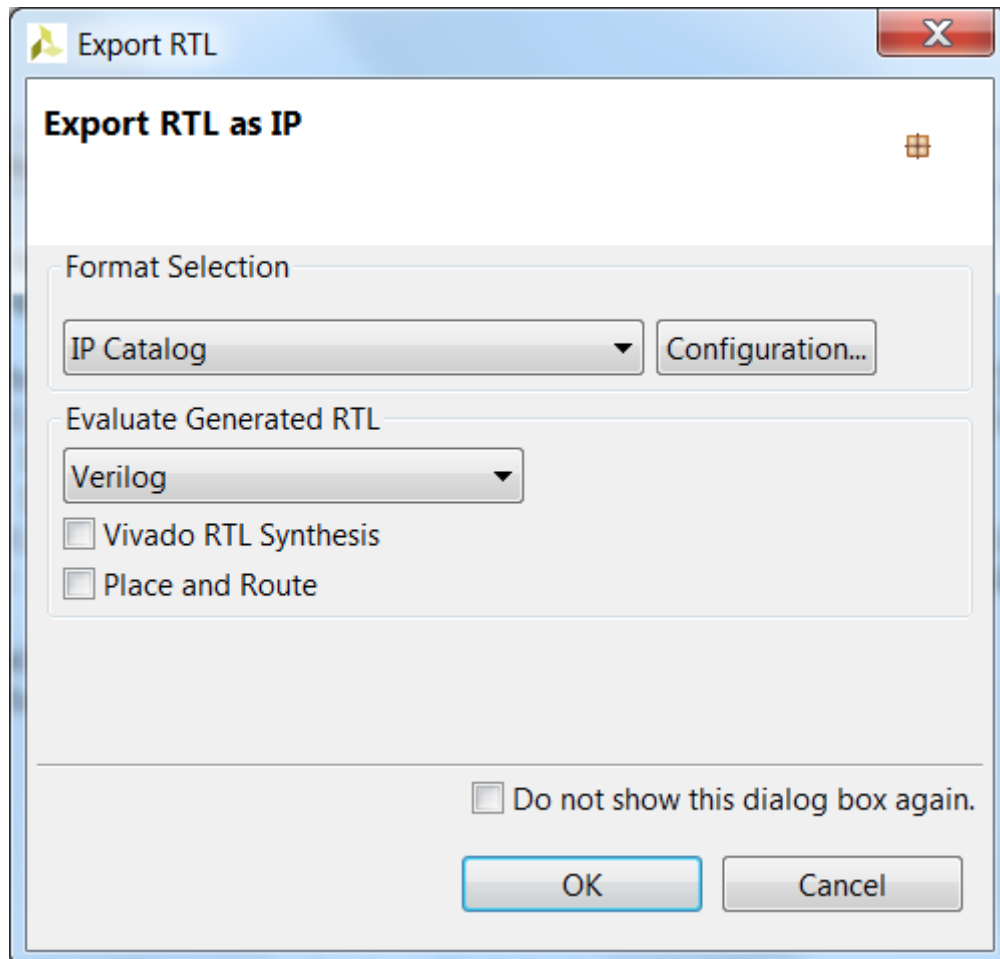
Observe that the design expects element data by providing `a_address0`, `a_ce0`, `b_address0`, `b_ce0` signals and outputs result using `res_d0`, `res_we0`, and `res_ce0`.

6. View various part of the simulation and try to understand how the design works.
7. When done, close Vivado by selecting **File > Exit**. Click **OK** if prompted, and then **Discard** to close the program without saving.

Export RTL and Implement

In Vivado HLS, export the design, selecting VHDL as a language, and run the implementation by selecting Evaluate option.

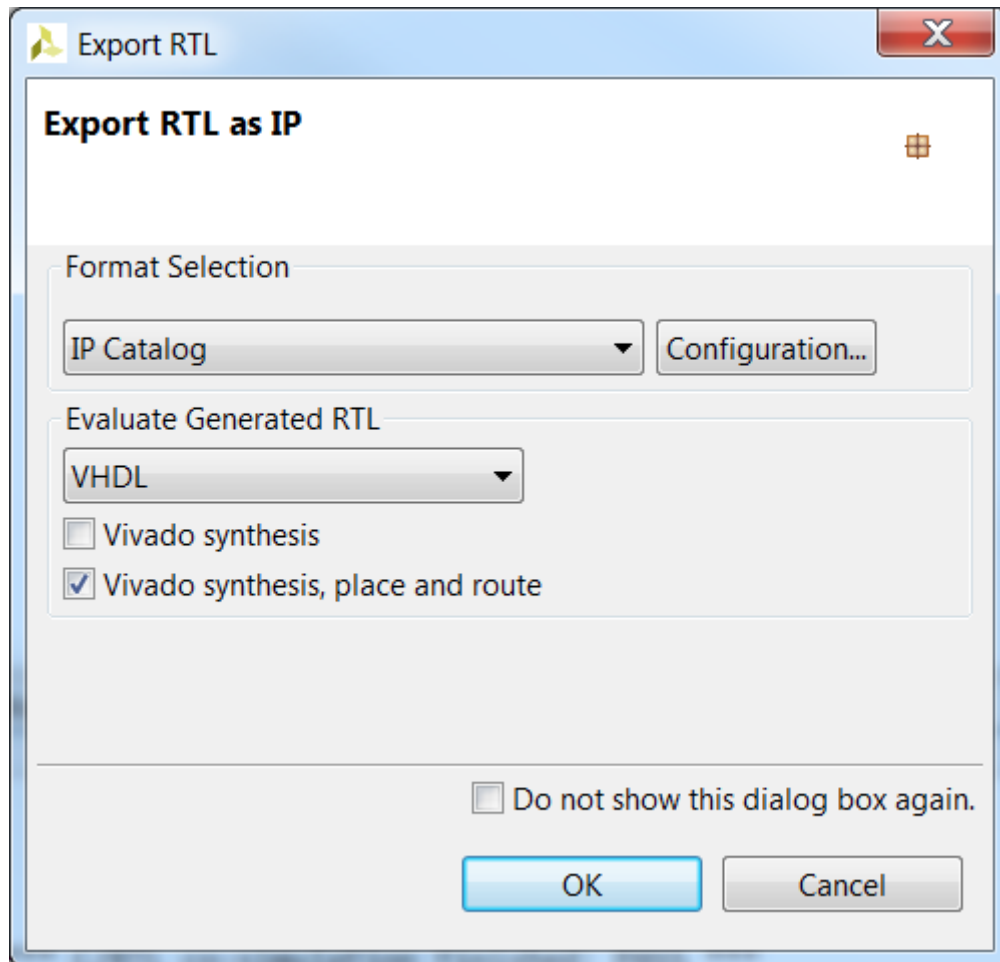
1. In Vivado-HLS, select **Solution > Export RTL** or click on the  button on tools bar to open the dialog box so the desired implementation can be run. An Export RTL Dialog box will open.



A Export RTL Dialog box

With default settings (shown above), the IP packaging process will run and create a package for the Vivado IP Catalog. Another option available from the Format Selection drop-down menu, is to create System Generator for DSP.

2. Click on the drop-down menu of the **Evaluate Generated RTL** field and select **VHDL**.
3. Click on the *Vivado synthesis, place and route* check box to run the implementation tool.



Selecting Evaluate options

4. Click **OK** and the implementation run will begin. You can observe the progress in the Vivado HLS Console window. It goes through several phases:

- Exporting RTL as an IP in the IP-XACT format
- RTL evaluation, since we selected Evaluate option 1. Goes through Synthesis 2. Goes through Placement and Routing

```
Vivado HLS Console
Implementation tool: Xilinx Vivado v.2018.2
Project:          matrixmul.prj
Solution:         solution1
Device target:    xc7z020clg400-1
Report date:      Mon Jul 30 19:19:25 +0800 2018

#=== Post-Implementation Resource usage ===
SLICE:           9
LUT:             33
FF:              27
DSP:             1
BRAM:            0
SRL:             0
#=== Final timing ===
CP required:     10.000
CP achieved post-synthesis:  3.016
CP achieved post-implementation:  2.652
Timing met
INFO: [Common 17-206] Exiting Vivado at Mon Jul 30 19:19:25 2018...
```

Console view

When the run is completed the implementation report will be displayed in the information pane.

Export Report for 'matrixmul'

General Information

Report date: Mon Jul 30 19:19:24 +0800 2018
 Project: matrixmul.prj
 Solution: solution1
 Device target: xc7z020clg400-1
 Implementation tool: Xilinx Vivado v.2018.2

Resource Usage

	VHDL
SLICE	9
LUT	33
FF	27
DSP	1
BRAM	0
SRL	0

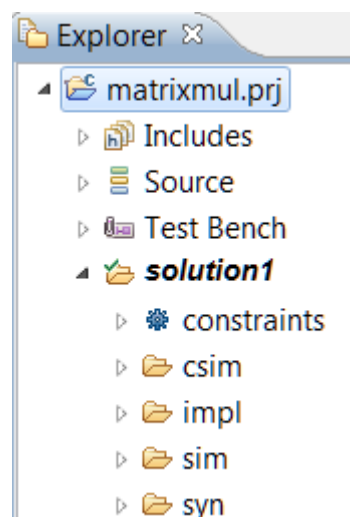
Final Timing

	VHDL
CP required	10.000
CP achieved post-synthesis	3.016
CP achieved post-implementation	2.652

Timing met

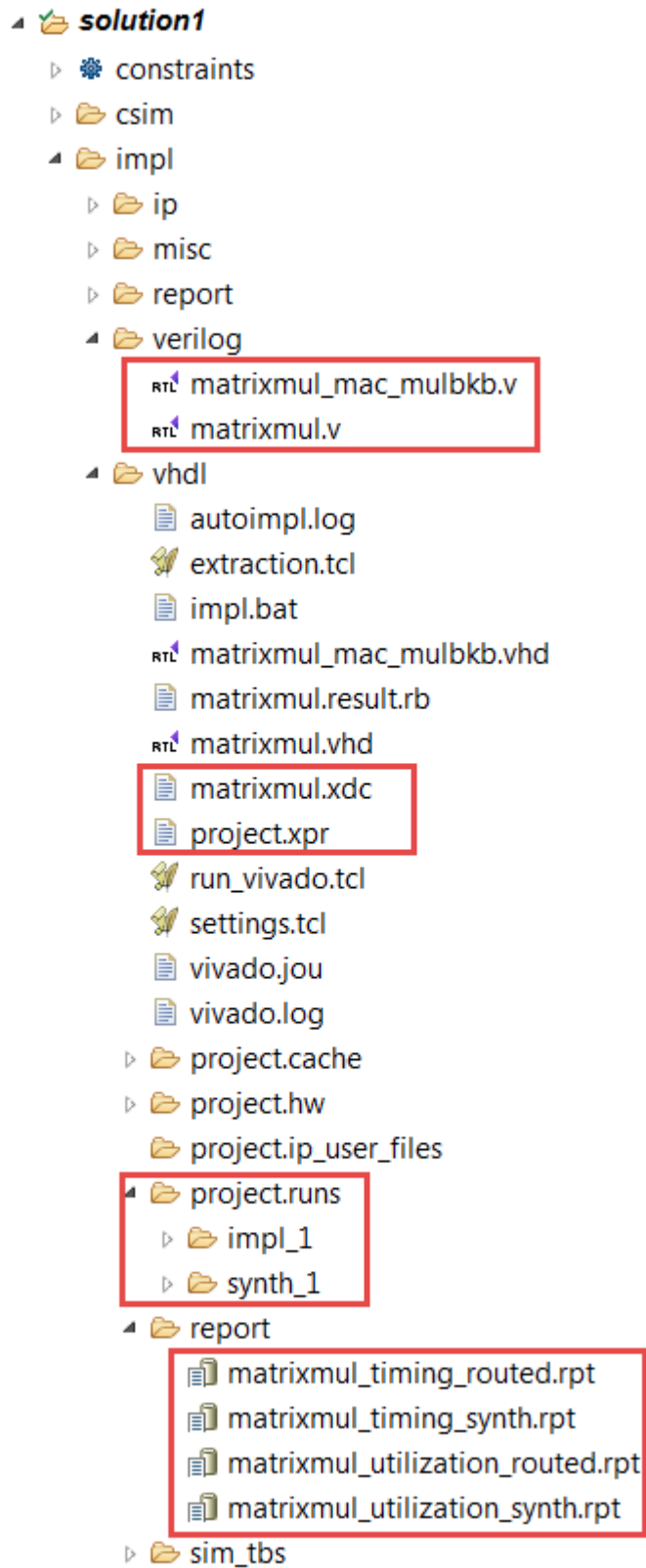
Implementation results in Vivado HLS

Observe that the timing constraint was met, the achieved period, and the type and amount of resources used.



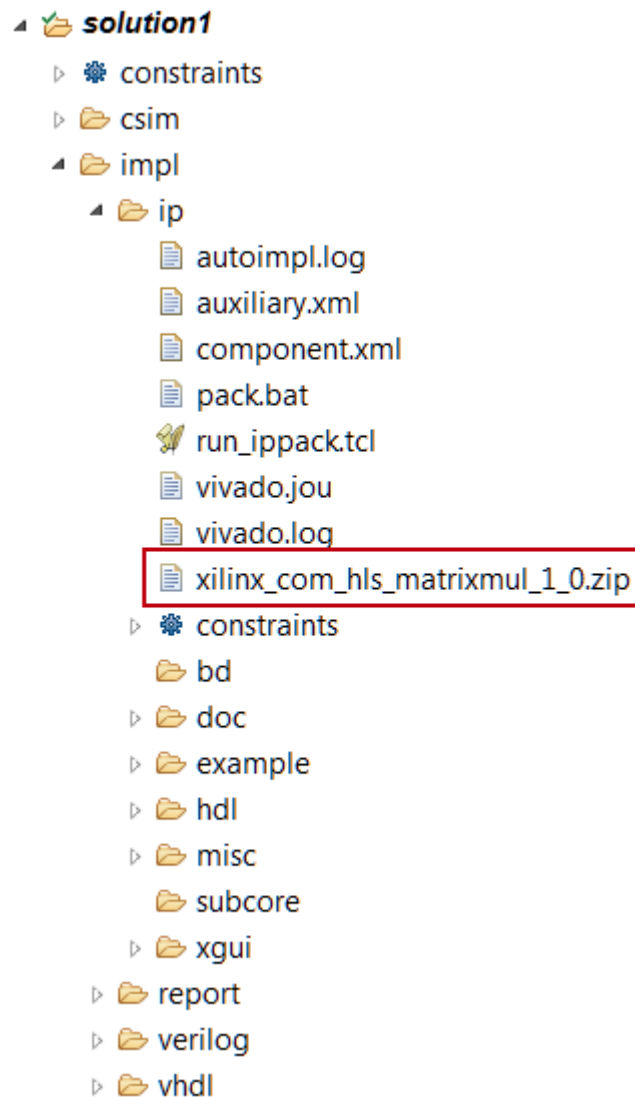
Explorer view after the RTL Export run

- Expand the Verilog and vhd1 sub-folders and observe that the Verilog sub-folder only has the rtl file whereas the vhd1 sub-folder has several files and sub-folders as the synthesis and implementation runs were made for it. It includes project.xpr file (the Vivado project file), matrixmul.xdc file (timing constraint file), project.runs folder (which includes synth_1 and impl_1 sub-folders created by the synthesis and implementation runs) among others.



The implementation directory

- Expand the **ip** folder and observe the IP packaged as a zip file, **xilinx_com_hls_matrixmul_1_0.zip**, which can be added to the Vivado IP catalog.



The ip folder content

7. Close Vivado HLS by selecting **File > Exit**.

Conclusion

In this lab, you completed the major steps of the high-level synthesis design flow using Vivado HLS. You created a project, adding source files, synthesized the design, simulated the design, and implemented the design. You also learned how to use the Analysis capability to understand the scheduling and binding.

Answers

Answers for question 1:

Estimated clock period: **8.70 ns**

Worst case latency: **79 clock cycles**

Number of DSP48E used: **1**

Number of FFs used: **44**

Number of LUTs used: **184**